

CHAPTER 8: INTRODUCTION TO OBJECT ORIENTED DEVELOPMENT

Object-Oriented Development (OOD) is a software development approach that focuses on designing and structuring software based on objects, which represent real-world entities by encapsulating both data (attributes) and behavior (methods).

❖ OBJECT-ORIENTED PARADIGM

Object-Oriented Paradigm is a design philosophy that mirrors the real world by representing entities as objects. It combines data and behavior into a single unit, making software easier to maintain, reuse, and scale.

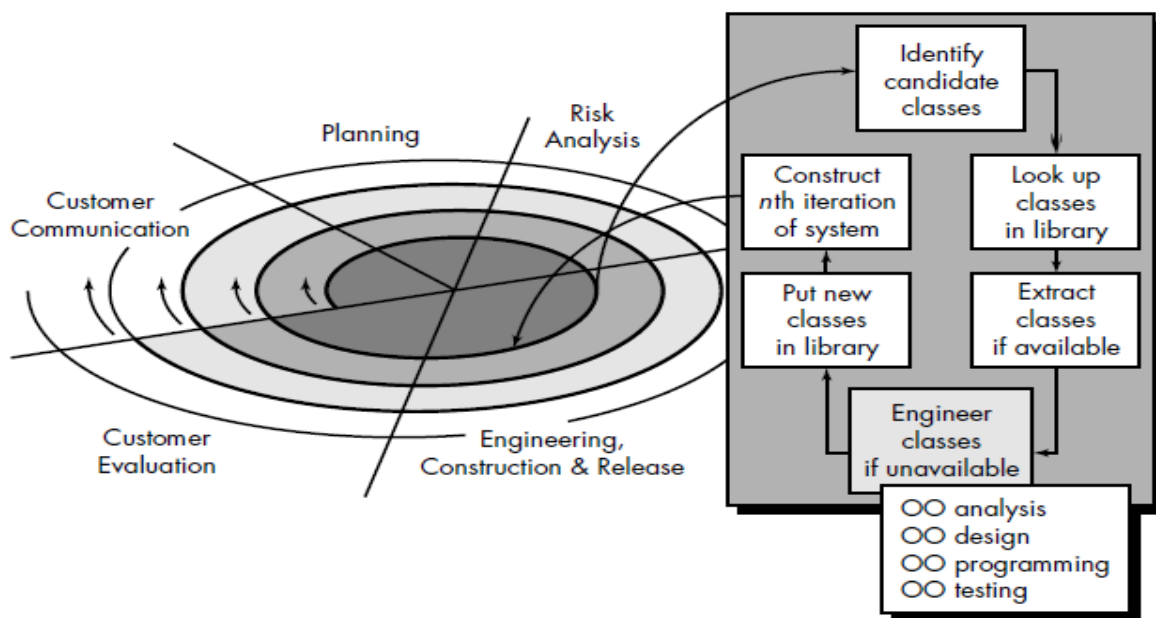
OO paradigm encompasses a complete view of software engineering. The benefits of object-oriented technology are enhanced if it is addressed early-on and throughout the software engineering process. Those considering object-oriented technology must assess its impact on the entire software engineering process. Merely employing object-oriented programming (OOP) will not yield the best results. Software engineers and their managers must consider such items as object-oriented requirements analysis (OORA), object-oriented design (OOD), object-oriented domain analysis (OODA), object-oriented database systems (OODBMS) and object-oriented computer aided software engineering (OOCASE).

OO systems are engineered using an evolutionary process model. An evolutionary process model, coupled with an approach that encourages component assembly (reuse), is the best paradigm for OO software engineering. Referring to Figure given below, the component-based development process model has been tailored for OO software engineering.

The OO process moves through an evolutionary spiral that starts with customer communication. It is here that the problem domain is defined and that basic problem classes are identified. Planning and risk analysis establish a foundation for the OO project plan. The technical work associated with OO software engineering follows the iterative path shown in the shaded box. OO software engineering emphasizes reuse. Therefore, classes are “looked up” in a library (of existing OO classes) before they are built. When a class cannot be found in the library, the software engineer applies object-oriented analysis (OOA), object-oriented design (OOD), object-oriented programming (OOP), and object-

oriented testing (OOT) to create the class and the objects derived from the class. The new class is then put into the library so that it may be reused in the future.

The object-oriented view demands an evolutionary approach to software engineering. As we will see that, it would be exceedingly difficult to define all necessary classes for a major system or product in a single iteration. As the OO analysis and design models evolve, the need for additional classes becomes apparent. It is for this reason that the paradigm just described works best for OO.



❖ OBJECT-ORIENTED CONCEPTS

Object-oriented = objects + classification + inheritance + communication

Object-oriented systems are built around the concepts of classes, objects, attributes, operations, messages, encapsulation, inheritance, and polymorphism. These concepts form the foundation of object-oriented analysis, design, and development.

1. Classes and Objects

A class is an OO concept that encapsulates the data and procedural abstractions required to describe the content and behavior of some real world entity. An object encapsulates both data (attributes) and the functions (operation, methods, or services) that manipulate the data.

A class is a generalized description (e.g., a template, pattern, or blueprint) that describes a collection of similar objects.

An object is an instance of a class that encapsulates both attributes (data) and operations (methods, functions, or services) that manipulate the data. By definition, all objects within a class inherit its attributes and operations, allowing uniform behavior while enabling multiple instances.

A superclass is a collection of classes, and a subclass is a specialized instance of a class. The data abstractions (attributes) that describe the class are enclosed by a “wall” of procedural abstractions (called operations, methods, or services) that are capable of manipulating the data in some way. The only way to reach the attributes (and operate on them) is to go through one of the methods that form the wall. Therefore, the class encapsulates data (inside the wall) and the processing that manipulates the data (the methods that make up the wall). This achieves information hiding and reduces the impact of side effects associated with change. Since the methods tend to manipulate a limited number of attributes, they are cohesive; and because communication occurs only through the methods that make up the “wall,” the class tends to be decoupled from other elements of a system. All of these design characteristics lead to high quality software.

2. Operations, Methods, and Services:

An object encapsulates data (represented as a collection of attributes) and the algorithms that process the data. These algorithms are called operations, methods, or services¹ and can be viewed as modules in a conventional sense. Each of the operations that are encapsulated by an object provides a representation of one of the behaviors of the object.

For example, the operation *GetColor* for the object **automobile** will extract the color stored in the color attribute. The implication of the existence of this operation is that the class **automobile** has been designed to receive a stimulus (we call the stimulus a *message*) that requests the color of the particular instance of a class. Whenever an object receives a stimulus, it initiates some behavior. This can be as simple as retrieving the color of automobile or as complex as the initiation of a chain of stimuli that are passed among a variety of different objects. In the latter case, consider an example in which the initial stimulus received by object 1 result in the generation of two other stimuli that are sent to

object 2 and object 3. Operations encapsulated by the second and third objects act on the stimuli, returning necessary information to the first object. Object 1 then uses the returned information to satisfy the behavior demanded by the initial stimulus.

3. Messages

Messages are the means by which objects interact. A message stimulates some behavior to occur in the receiving object. The behavior is accomplished when an operation is executed. An object is requested to perform one of its operations by sending it a message telling the object what to do. The receiver [object] responds to the message by first choosing the operation that implements the message name, executing this operation, and then returning control to the caller.

Messages and methods [operations] are two sides of the same coin. Methods are the procedures that are invoked when an object receives a message.

The interaction between messages is illustrated schematically in Figure below. An operation within a sender object generates a message of the form

Message: [destination, operation, parameters]

where *destination* defines the *receiver object* that is stimulated by the message, *operation* refers to the operation that is to receive the message, and *parameters* provides information that is required for the operation to be successful.

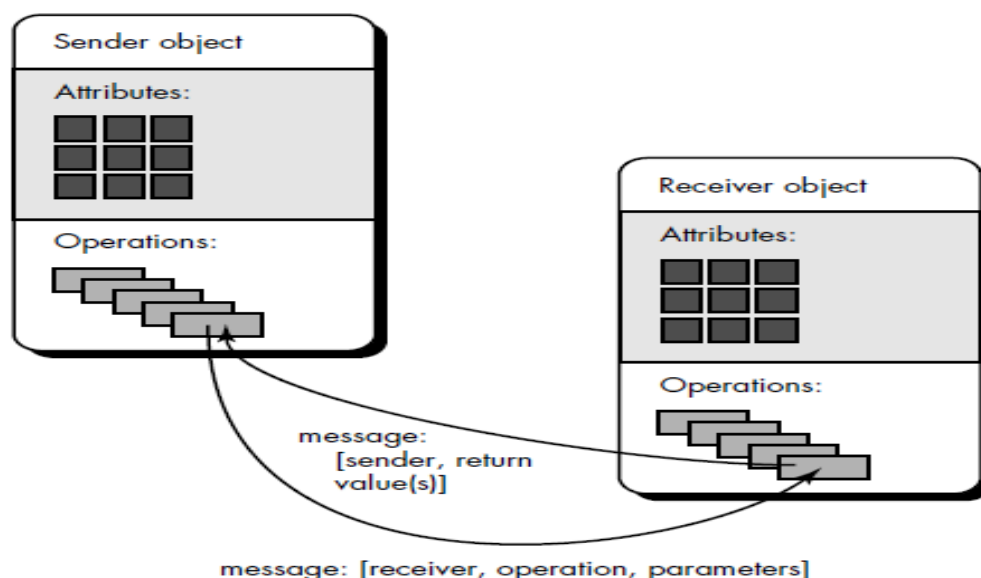


Figure: message passing between objects

Considering the objects shown in figure below an example of message passing within an OO system can be demonstrated. Four objects, **A**, **B**, **C**, and **D** communicate with one another

by passing messages. For example, if object **B** requires processing associated with operation *op10* of object **D**, it would send **D** a message of the form message: [**D**, *op10*, {data}]

As part of the execution of *op10*, object **D** may send a message to object **C** of the form

message: (**C**, *op08*, {data})

Then **C** finds *op08*, performs it, and sends an appropriate return value to **D**. Operation *op10* completes and sends a return value to **B**.

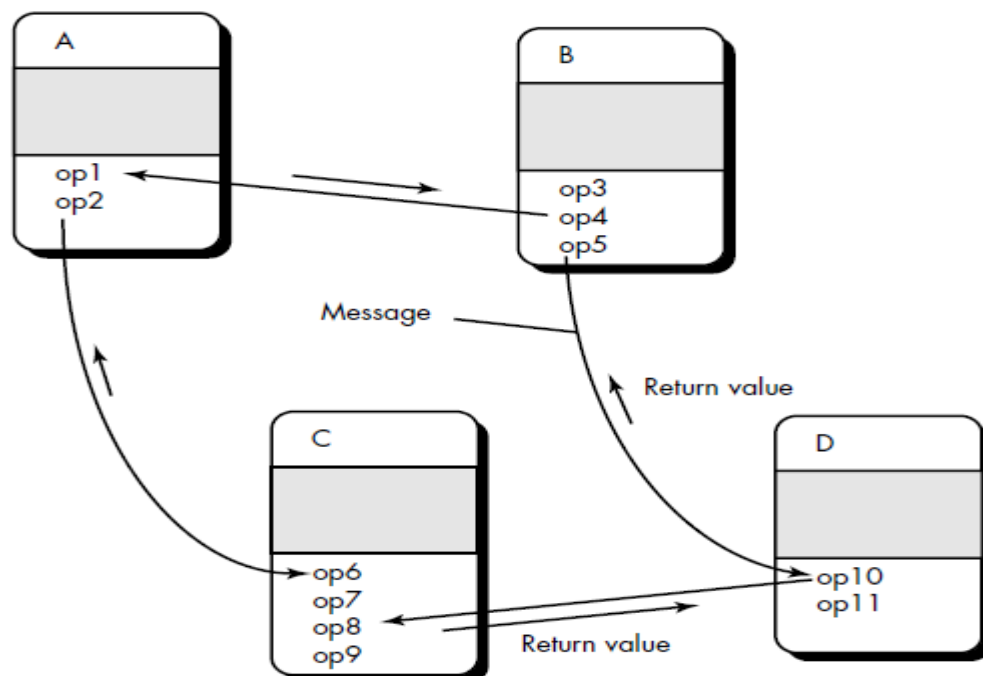


fig: Message passing example

4. Abstraction

Abstraction involves hiding the complex implementation details and showing only the essential features of the object. This helps in managing complexity by allowing the designer to focus on the interactions at a higher level.

Example:

Driving a car: you use start(), stop(), accelerate() without knowing the engine mechanics.

Types:

- **Data abstraction** refers to the mechanism of hiding the internal representation of data within a class while exposing only the necessary operations that allow controlled access and modification.
- **Procedural abstraction** involves hiding the internal implementation of methods (operations) while exposing only their interface. It allows objects to perform operations without revealing the underlying computational details.

5. Encapsulation

Encapsulation is a core OOP principle that bundles data (variables) and the methods (functions) acting on that data into a single unit, typically a class. It restricts direct access to some component's data, protecting it from accidental corruption and unauthorized modification, which improves data integrity and maintainability.

Example

Think of a car as an object. The internal workings (engine, fuel injection) are encapsulated within the car's body. The driver (user) interacts with the car only through a defined interface (steering wheel, pedals, dashboard), without needing to understand or directly modify the internal mechanics.

6. Inheritance

Inheritance provides a means for allowing subclasses to reuse existing superclass data and procedures; also provides mechanism for propagating changes

A subclass **Y** inherits all of the attributes and operations associated with its superclass, **X**. This means that all data structures and algorithms originally designed and implemented for **X** are immediately available for **Y**—no further work need be done. Reuse has been accomplished directly. Any change to the

data or operations contained within a superclass is immediately inherited by all subclasses that have inherited from the superclass. Therefore, the class hierarchy becomes a mechanism through which changes (at high levels) can be immediately propagated through a system. It is important to note that, at each level of the class hierarchy, new attributes and operations may be added to those that have been inherited from higher levels in the hierarchy.

7. Polymorphism

Polymorphism is a characteristic that greatly reduces the effort required to extend an existing OO system.

Polymorphism is a mechanism that allows several objects in a class hierarchy to have different methods with the same name (instances of each subclass will be free to respond to messages by calling their own version of the method)

Polymorphism enables a number of different operations to have the same name. This in turn decouples objects from one another, making each more independent.

The ability of different objects to respond, each in its own way, to identical messages is called *polymorphism*.

Polymorphism results from the fact that every class lives in its own name space. The names assigned within a class definition won't conflict with names assigned anywhere outside it. This is true both of the instance variables in an object's data structure and of the object's methods:

- Just as the fields of a C structure are in a protected name space, so are an object's instance variables.
- Method names are also protected. Unlike the names of C functions, method names aren't global symbols. The name of a method in one class can't conflict with method names in other classes; two very different classes could implement identically named methods.

The main benefit of polymorphism is that it simplifies the programming interface. It permits conventions to be established that can be reused in class after class. Instead of inventing a new name for each new function you add to a program, the same names can be reused.

❖ IDENTIFYING THE ELEMENTS OF AN OBJECT MODEL

The elements of an object model—classes and objects, attributes, operations, and messages were each defined and discussed in the preceding section. But how do we go about identifying these elements for an actual problem?

The following sections present a series of informal guidelines that will assist in the identification of the elements of the object model.

- 1. Analyze System Requirements:** Understand the problem domain by reviewing the problem statement or requirements documents to identify all the necessary system functionalities.
- 2. Identifying Classes and Objects:** We can begin to identify objects by examining the problem statement or by performing a "grammatical parse" on the processing narrative for the system to be built. Objects are determined by underlining each noun or noun clause and entering it in a simple table. Objects can be:
 - *External entities* (e.g., other systems, devices, people) that produce or consume information to be used by a computer-based system.
 - *Things* (e.g., reports, displays, letters, signals) that are part of the information domain for the problem.
 - *Occurrences or events* (e.g., a property transfer or the completion of a series of robot movements) that occur within the context of system operation.
 - *Roles* (e.g., manager, engineer, salesperson) played by people who interact with the system.
 - *Organizational units* (e.g., division, group, and team) those are relevant to an application.
 - *Places* (e.g., manufacturing floor or loading dock) that establish the context of the problem and the overall function of the system.
 - *Structures* (e.g., sensors, four-wheeled vehicles, or computers) that define a class of objects or in the extreme, related classes of objects.
- 3. Specifying Attributes:** Attributes are chosen by examining the problem statement, looking for things that fully define an object and make it unique. Attributes describe an object that has been selected for inclusion in the analysis model.

In essence, it is the attributes that define the object—that clarify what is meant by the object in the context of the problem space. For example, if

we were to build a system that tracks baseball statistics for professional baseball players, the attributes of the object **player** would be quite different than the attributes of the same object when it is used in the context of the professional baseball pension system. In the former, attributes such as name, position, batting average, fielding percentage, years played, and games played might be relevant. For the latter, some of these attributes would be meaningful, but others would be replaced (or augmented) by attributes like average salary, credit toward full vesting, pension plan options chosen, mailing address, and the like.

To develop a meaningful set of attributes for an object, the analyst can again study the processing narrative (or statement of scope) for the problem and select those things that reasonably "belong" to the object.

4. **Defining Operations:** Operations define the behavior of an object and change the object's attributes in some way. More specifically, an operation changes one or more attribute values that are contained within the object. Therefore, an operation must have "knowledge" of the nature of the objects attributes and must be implemented in a manner that enables it to manipulate the data structures that have been derived from the attributes. Although many different types of operations exist, they can generally be divided into three broad categories: (1) operations that manipulate data in some way (e.g., adding, deleting, reformatting, selecting), (2) operations that perform a computation, and (3) operations that monitor an object for the occurrence of a controlling event.
5. **Identify relationships:** Identifying relationships is the process of determining how the objects in the object model are connected, associated, or dependent on one another. It defines the structural links between objects and explains how they collaborate to achieve system functionality. The main types of relationships in an object model are **association, aggregation, composition, inheritance (generalization), and dependency**.
6. **Finalizing the Object Definition:** The definition of operations is the last step in completing the specification of an object. Operations were culled from a grammatical parse of the processing narrative for the system. Additional operations may be determined by considering the "life history"

of an object and the messages that are passed among objects defined for the system.

The generic life history of an object can be defined by recognizing that the object must be created, modified, manipulated or read in other ways, and possibly deleted. For the **system** object, this can be expanded to reflect known activities that occur during its life. Some of the operations can be ascertained from likely communication between objects.

❖ DOMAIN ANALYSIS

Domain Analysis is the process of studying and understanding the problem domain (real-world area in which the system will operate) to identify common concepts, objects, relationships, and rules before designing the software system. Analysis for object-oriented systems can occur at many different levels of abstraction. OOA at a middle level of abstraction is called as *domain analysis*. *Domain analysis* is performed when an organization wants to create a library of reusable classes (components) that will be broadly applicable to an entire category of applications.

Main Objective of domain analysis is to define a set of classes (objects) that are encountered throughout an application domain. These can then be reused in many applications.

• Reuse and Domain Analysis

Object-technologies are leveraged through reuse. Benefits derived from reuse are consistency and familiarity. Patterns within the software will become more consistent, leading to better maintainability. Be certain to establish a set of reuse design rules” so that these benefits are achieved.

Reuse is the cornerstone of component-based software engineering.

• Domain Analysis Process

Software domain analysis is the identification, analysis, and specification of common requirements from a specific application domain, typically for reuse on multiple projects within that application domain.

Object-oriented domain analysis is the identification, analysis, and specification of common, reusable capabilities within a specific application domain, in terms of common objects, classes, subassemblies, and frameworks.

The goal of domain analysis is straightforward: to find or create those classes that are broadly applicable, so that they may be reused.

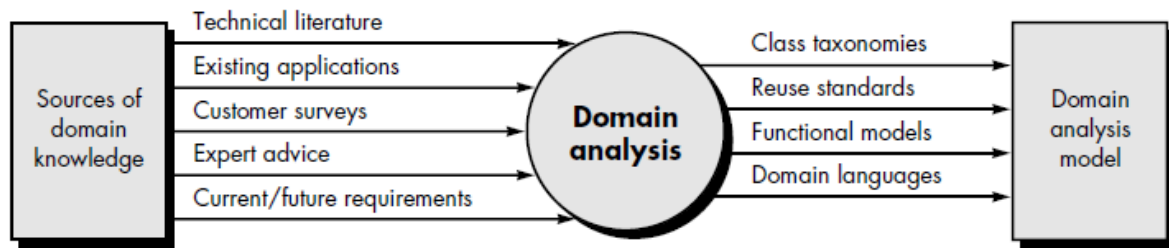


Fig: Input and Output for domain analysis

Domain analysis may be viewed as an umbrella activity for the software process. By this we mean that domain analysis is an ongoing software engineering activity that is not connected to any one software project. The role of the domain analyst is to design and build reusable components that may be used by many people working on similar but not necessarily the same applications.

Above figure illustrates key inputs and outputs for the domain analysis process. Sources of domain knowledge are surveyed in an attempt to identify objects that can be reused across the domain. In essence domain analysis is quite similar to knowledge engineering. The knowledge engineer investigates a specific area of interest in an attempt to extract key facts that may be of use in creating an expert system or artificial neural network. During domain analysis, *object* (and class) *extraction* occurs.

The domain analysis process can be characterized by a series of activities that begin with the identification of the domain to be investigated and end with a specification of the objects and classes that characterize the domain. Berard suggests the following activities:

- **Define the domain to be investigated.** To accomplish this, the analyst must first isolate the business area, system type, or product category of interest. Next, both OO and non-OO “items” must be extracted. OO items include specifications, designs, and code for existing OO application classes; support classes (e.g., GUI classes or database access classes); commercial off-the shelf (COTS) component libraries that are relevant to the domain; and test cases. Non-OO items encompass policies, procedures, plans, standards, and guidelines; parts of existing non-OO applications (including specification, design, and test information); metrics; and COTS non-OO software.
- **Categorize the items extracted from the domain.** The items are organized into categories and the general defining characteristics of the category are

defined. A classification scheme for the categories is proposed and naming conventions for each item are defined. When appropriate, classification hierarchies are established.

- **Collect a representative sample of applications in the domain.** To accomplish this activity, the analyst must ensure that the application in question has items that fit into the categories that have already been defined. During the early stages of use of object-technologies, a software organization will have few if any OO applications. Therefore, the domain analyst must “identify the conceptual (as opposed to physical) objects in each [non-OO] application.”
- **Analyze each application in the sample.** The following steps are followed by the analyst:
 - Identify candidate reusable objects.
 - Indicate the reasons that the object has been identified for reuse.
 - Define adaptations to the object that may also be reusable.
 - Estimate the percentage of applications in the domain that might make reuse of the object.
 - Identify the objects by name and use configuration management techniques to control them. In addition, once the objects have been defined, the analyst should estimate what percentage of a typical application could be constructed using the reusable objects.
- **Develop an analysis model for the objects.** The analysis model will serve as the basis for design and construction of the domain objects.

❖ COMPONENTS OF THE OO ANALYSIS MODEL

To develop a “precise, concise, understandable, and correct model of the real world,” a software engineer must select a notation that implements a set of generic components of an OO analysis model. Monarchi and Puhr define a set of generic representational components that appear in all OO analysis models.

1. **Static components** are structural in nature and indicate characteristics that hold throughout the operational life of an application. These characteristics distinguish one object from other objects.
2. **Dynamic components** focus on control and are sensitive to timing and event processing. They define how one object interacts with other objects over time.

Static components do not change as the application is executed. Dynamic components are influenced by timing and events.

The following components are identified:

- **Static view of semantic classes.** Requirements are assessed and classes are extracted (and represented) as part of the analysis model. These classes persist throughout the life of the application and are derived based on the semantics of the customer requirements.
- **Static view of attributes.** Every class must be explicitly described. The attributes associated with the class provide a description of the class, as well as a first indication of the operations that are relevant to the class.
- **Static view of relationships.** Objects are “connected” to one another in a variety of ways. The analysis model must represent these relationships so that operations (that affect these connections) can be identified and the design of a messaging approach can be accomplished.
- **Static view of behaviors.** The relationships just noted define a set of behaviors that accommodate the usage scenario (use-cases) of the system. These behaviors are implemented by defining a sequence of operations that achieve them.
- **Dynamic view of communication.** Objects must communicate with one another and do so based on a series of events that cause transition from one state of a system to another.
- **Dynamic view of control and time.** The nature and timing of events that cause transitions among states must be described.

❖ THE OOA PROCESS

Object-Oriented Analysis (OOA) is the process of understanding system requirements by identifying relevant real-world objects, their attributes, behaviors, and relationships, without considering how the system will be implemented.

The OOA process does not begin with a concern for objects. Rather, it begins with an understanding of the manner in which the system will be used—by people, if the system is human-interactive; by machines, if the system is involved in process control; or by other programs, if the system coordinates and controls applications. Once the scenario of usage has been defined, the modeling of the software begins. The sections that follow define a series of techniques that may be used to gather basic customer requirements and then define an analysis model for an object oriented system.

Major activities/ components:

1. Use Cases

Use-cases are an excellent requirements elicitation tool, regardless of the analysis method that is used.

Use-cases model the system from the end-user's point of view. Created during requirements elicitation, use-cases should achieve the following objectives:

- To define the functional and operational requirements of the system (product) by defining a scenario of usage that is agreed upon by the end-user and the software engineering team.
- To provide a clear and unambiguous description of how the end-user and the system interact with one another.
- To provide a basis for validation testing.

During OOA, use-cases serve as the basis for the first element of the analysis model.

Using UML notation, a diagrammatic representation of a use-case, called a *use-case diagram*, can be created. Like many elements of the analysis model, the use-case diagram can be represented at many levels of abstraction. The use-case diagram contains actors and use-cases. *Actors* are entities that interact with the system. They can be human users or other machines or systems that have defined interfaces to the software.

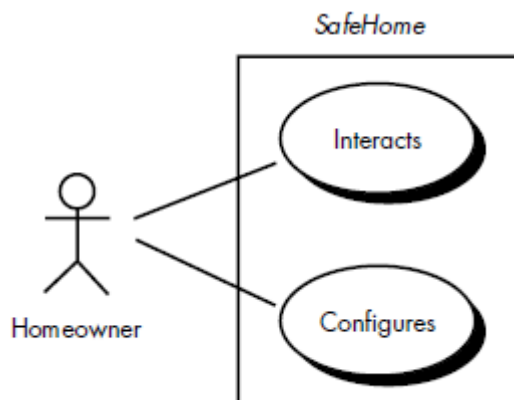


Fig: High level Use-case

2. Class-Responsibilities-Collaborator Modeling

Once basic usage scenarios have been developed for the system, it is time to identify candidate classes and indicate their responsibilities and collaborations. *Class-Responsibility- Collaborator* (CRC) modeling provides a simple means for identifying and organizing the classes that are relevant to system or product requirements.

A CRC model is really a collection of standard index cards that represent classes. The cards are divided into three sections. Along the top of the card you write the

name of the class. In the body of the card you list the class responsibilities on the left and the collaborators on the right.

CRC model may make use of actual or virtual *index cards*. The intent is to develop an organized representation of classes. *Responsibilities* are the attributes and operations that are relevant for the class. Stated simply, a responsibility is “anything the class knows or does”. *Collaborators* are those classes that are required to provide a class with the information needed to complete a responsibility. In general, collaboration implies either a request for information or a request for some action.

Class name:	
Class type: (e.g., device, property, role, event)	
Class characteristic: (e.g., tangible, atomic, concurrent)	
responsibilities:	collaborations:

Fig: A CRC model Index card

3. Define structure and hierarchies

Once classes and objects have been identified using the CRC model, the analyst begins to focus on the structure of the class model and the resultant hierarchies that arise as classes and subclasses emerge. Using UML notation, a variety of class diagrams can be created. *Generalization/specialization* class structures can be created for identified classes.

Structure representations provide the analyst with a means for partitioning the CRC model and representing that partitioning graphically. The expansion of each class provides needed detail for review and for subsequent design.

4. Define Subjects and Subsystem

An analysis model for a complex application may have hundreds of classes and dozens of structures. For this reason, it is necessary to define a concise representation that is a digest of the CRC and structure models just described. When a group of all classes collaborate among themselves to accomplish a set of cohesive responsibilities, they are often referred to as *subsystems* or *packages* (in UML terminology). Subsystems or packages are abstractions that provide a reference or pointer to more detail in the analysis model. When viewed from the

outside, a subsystem can be treated as a black box that contains a set of responsibilities and that has its own (outside) collaborators. A subsystem implements one or more *contracts* with its outside collaborators. A contract is a specific list of requests that collaborators can make of the subsystem. Subsystems can be represented with the context of CRC modeling by creating a subsystem index card. The subsystem index card indicates the name of the subsystem, the contracts that the subsystem must accommodate, and the classes or (other) subsystems that support the contract. Packages are identical to subsystems in intent and content but are represented graphically in UML.

❖ OBJECT-ORIENTED DESIGN

Object-oriented design transforms the analysis model created using object-oriented analysis into a design model that serves as a blueprint for software construction. The design of object-oriented software requires the definition of a multilayered software architecture, the specification of subsystems that perform required functions and provide infrastructure support, a description of objects (classes) that form the building blocks of the system, and a description of the communication mechanisms that allow data to flow between layers, subsystems, and objects.

OOD is a process of creating a well-structured and Organized Design for a Software System based on Object Oriented Paradigm and it is Important part in System Design. Object Oriented Design focuses on the implementation of System based on Analysis and it involves taking the models and insights gathered during the analysis phase and turning out into well-structured, efficient software system.

Object Oriented Design includes both:

- System Design
- Object Design

- **System design:**

System Design is critical part in Object Oriented Design where we design a Software System based on the information gathered during Analysis of the requirement of the Software. System Design focuses on the roadmap that provides how various components and modules of the software will work together. Designing a system is done according to both Analysis and the purposed Architecture of system

- **Object Design:**

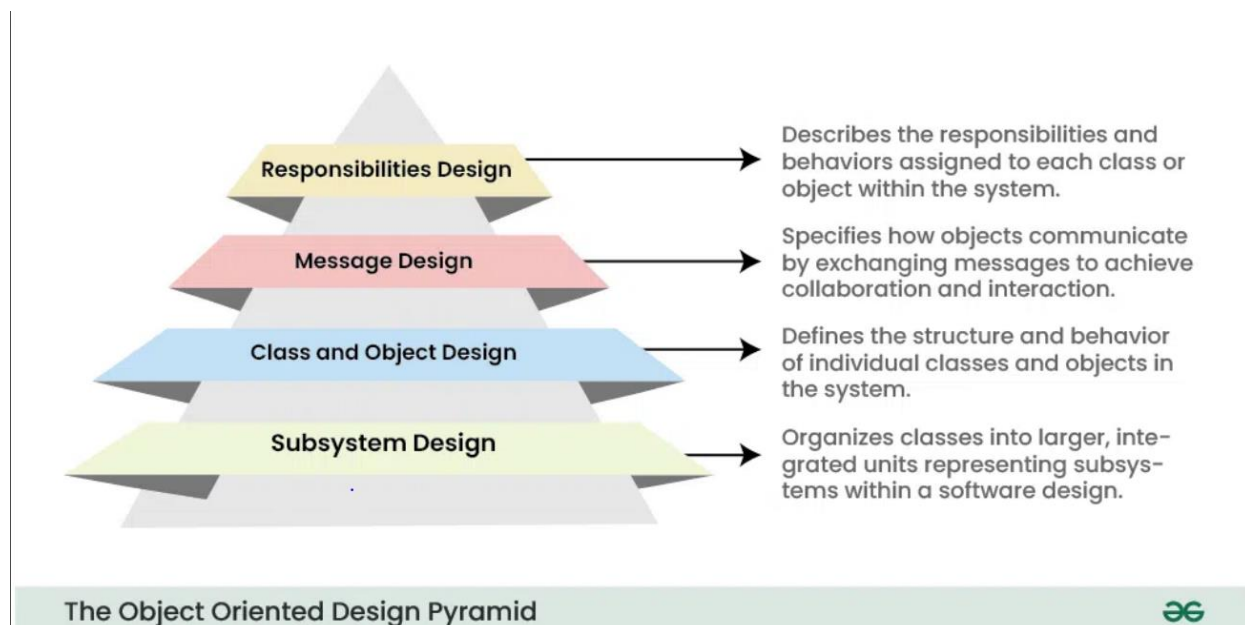
Object Design is critical part of Object-Oriented Design where the main focus is on the detailed design of individual Classes and Objects, Specifying how they are

Implemented on the System design of Software Development. Object Design is the foundation of writing Actual Code for the Software Development and it must be ensured that system is efficient, easily maintainable and extensible.

❖ DESIGN FOR OBJECT-ORIENTED SYSTEMS

For object-oriented systems, we can also define a design pyramid, the four layers of the OO design pyramid are:

- **The subsystem layer** contains a representation of each of the subsystems that enable the software to achieve its customer-defined requirements and to implement the technical infrastructure that supports customer requirements.
- **The class and object layer** contains the class hierarchies that enable the system to be created using generalizations and increasingly more targeted specializations. This layer also contains representations of each object.
- **The message layer** contains the design details that enable each object to communicate with its collaborators. This layer establishes the external and internal interfaces for the system.
- **The responsibilities layer** focuses on the responsibilities of individual objects. This includes defining the behavior of each class, specifying what each object is responsible for, and how it responds to messages.



The design pyramid focuses exclusively on the design of a specific product or system. It should be noted, however, that another “layer” of design exists, and

this layer forms the foundation on which the pyramid rests. The foundation layer focuses on the design of domain objects (called design patterns). Domain objects play a key role in building the infrastructure for the OO system by providing support for human/computer interface activities, task management, and data management. Domain objects can also be used to flesh out the design of the application itself.

Translating an OOA model into an OOD model

Figure Below illustrates the relationship between the OO analysis model and design model that will be derived from it. The subsystem design is derived by considering overall customer requirements (represented with use-cases) and the events and states that are externally observable (the object-behavior model). Class and object design is mapped from the description of attributes, operations, and collaborations contained in the CRC model. Message design is driven by the object-relationship model, and responsibilities design is derived using the attributes, operations, and collaborations described in the CRC model.

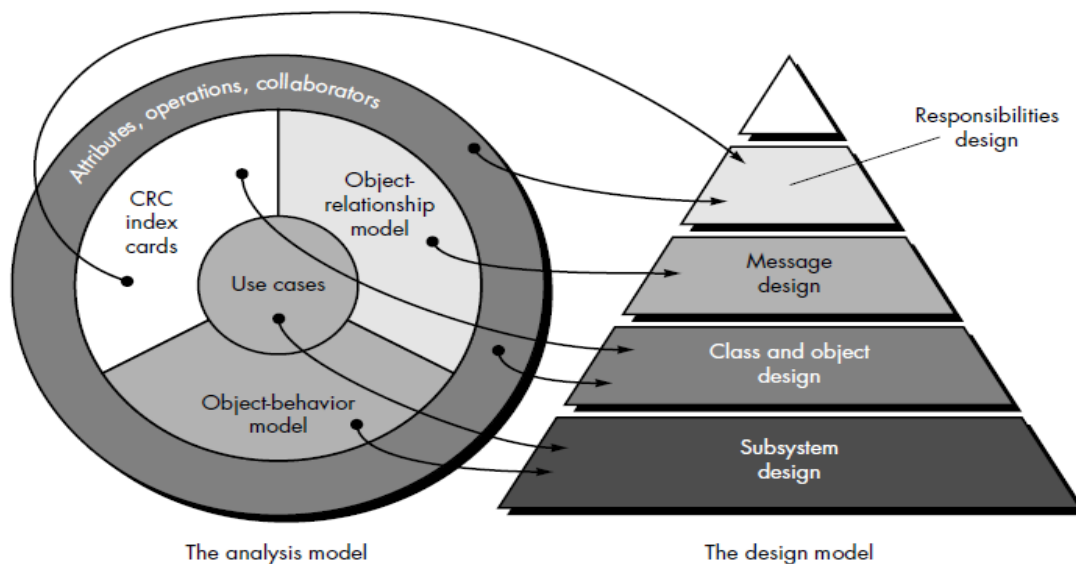


Fig: Translating OOA to OOD

❖ THE SYSTEM DESIGN PROCESS

In Object-Oriented Design (OOD), the system design process is a high-level design activity that follows Object-Oriented Analysis (OOA) and precedes object design. The primary purpose of system design is to define the overall architecture and structure of the system in a manner that satisfies both functional and non-functional requirements.

System design focuses on identifying subsystems, architectural styles, deployment strategies, and global control mechanisms, providing a framework within which detailed object design can be carried out.

Key Activities in the System Design Process

- **System partitioning** involves dividing a complex software system into smaller, manageable subsystems or layers. This activity is essential for organizing the system into logical components such as the presentation layer, business logic layer, and data layer. By partitioning the system, developers can focus on individual modules independently, which improves modularity, maintainability, and simplifies the integration of subsystems into the overall architecture.
- **Concurrency design** is the process of identifying tasks within the system that can be executed simultaneously and determining how to coordinate these tasks. This activity is crucial for enhancing system efficiency and responsiveness, particularly in applications where multiple processes occur concurrently. Proper concurrency design ensures that tasks are synchronized correctly, preventing conflicts and maintaining system consistency.
- **Hardware and software mapping** defines how system components will be allocated to specific hardware resources and which software platforms will be used, including operating systems, databases, and middleware. This activity ensures optimal utilization of resources, system compatibility, and efficient performance. Proper mapping reduces system bottlenecks and supports scalability and reliability in distributed or complex environments.
- **Data management design** involves specifying how data will be stored, accessed, and organized within the system. Decisions include selecting appropriate storage mechanisms, such as relational databases or flat files, and defining efficient methods for data retrieval and update. This activity is important for maintaining data integrity, consistency, and system efficiency, particularly in data-intensive applications.

- **Global resource management** addresses the control and coordination of resources shared among multiple system components, such as memory, network bandwidth, or global variables. Effective management of these resources is critical to avoid conflicts, ensure synchronization, and maintain overall system reliability. It is especially important in distributed systems or applications with concurrent processes.
- **Software control design** determines how the execution of system operations will be managed. This includes selecting an appropriate control mechanism, such as event-driven control for graphical user interfaces or procedural control for sequential task execution. Proper control design ensures predictable system behavior, responsiveness to user inputs or external events, and smooth operation across all system components.

❖ THE OBJECT DESIGN PROCESS

Object design is concerned with the detailed design of the objects and their interactions. It is completed within the overall architecture defined during system design and according to agreed design guidelines and protocols. Object design is particularly concerned with the specification of attribute types, how operations function, and how objects are linked to other objects.

It is at this stage that the basic concepts and principles associated with component level design come into play. Local data structures are defined (for attributes) and algorithms (for operations) are designed.

Key components:

1. Object Descriptions

A design description of an object (an instance of a class or subclass) can take one of two forms:

- **Protocol description** that establishes the interface of an object by defining each message that the object can receive and the related operation that the object performs when it receives the message.
- **Implementation description** that shows implementation details for each operation implied by a message that is passed to an object. Implementation details include information about the object's private part; that is, internal details about the data structures that describe the object's attributes and procedural details that describe operations. The protocol description is nothing more than a set of messages and a corresponding comment for each message.

2. Designing Algorithms and Data Structures

A variety of representations contained in the analysis model and the system design provide a specification for all operations and attributes. Algorithms and data structures are designed using an approach that differs little from the data design and component-level design approaches discussed for conventional software engineering. An algorithm is created to implement the specification for each operation. In many cases, the algorithm is a simple computational or procedural sequence that can be implemented as a self-contained software module. However, if the specification of the operation is complex, it may be necessary to modularize the operation. Conventional component-level design techniques can be used to accomplish this. Data structures are designed concurrently with algorithms. Since operations invariably manipulate the attributes of a class, the design of the data structures that best reflect the attributes will have a strong bearing on the algorithmic design of the corresponding operations.

Although many different types of operations exist, they can generally be divided into three broad categories:

- (1) operations that manipulate data in some way (e.g., adding, deleting, reformatting, selecting),
- (2) operations that perform a computation, and
- (3) operations that monitor an object for the occurrence of a controlling event.

3. Program Components and Interfaces

An important aspect of software design quality is modularity; that is, the specification of program components (modules) that are combined to form a complete program. The object-oriented approach defines the object as a program component that is itself linked to other components (e.g., private data, operations). But defining objects and operations is not enough. During design, we must also identify the interfaces between objects and the overall structure (considered in an architectural sense) of the objects. Although a program component is a design abstraction, it should be represented in the context of the programming language used for implementation. To accommodate OOD, the programming language to be used for implementation should be capable of creating the following program component.

Difference between object-oriented analysis and object-oriented design (IMP)

Object-Oriented Analysis (OOA)	Object-Oriented Design (OOD)
Concerned with understanding the problem and its requirements.	Focuses on transforming requirements into a detailed design.
Involves gathering and defining what the software should do.	Involves how the software will do what is needed.
Emphasizes capturing real-world concepts into a software model.	Emphasizes translating the captured model into a technical solution.
Primarily uses techniques like use cases, class diagrams, and behavior diagrams to understand the problem domain.	Involves architecture, detailed class design, object interactions, and design patterns.
Identifies entities, their relationships, and their interactions in the problem domain.	Refined identified entities into detailed classes, their attributes, methods, and interactions.
Focuses on stakeholder's needs, domain understanding, and requirements gathering.	Focuses on implementation details, coding strategies, and testing plans.
Provides the foundation for creating a model of the problem domain.	Provides the foundation for the implementation and coding of the solution.